

数字逻辑设计

课程设计实验报告

基于 FPGA 的弹幕射击游戏

Danmaku Shooting Game on FPGA

课程名称	数字逻辑设计
实验平台	Sword Kintex-7 (XC7K160TFFG676-1)
开发环境	Xilinx Vivado 2025.1
硬件描述语言	Verilog HDL
提交日期	2026 年 7 月

学号	姓名	贡献比例
[学号 1]	[姓名 1]	x%
[学号 2]	[姓名 2]	y%
[学号 3]	[姓名 3]	z%

目录

1	背景介绍	3
2	设计说明	3
2.1	设计开发环境	3
2.2	输入输出交互选择	3
2.2.1	键盘输入	3
2.2.2	显示输出	4
2.3	系统总体架构	4
2.4	引脚约束	5
2.5	时钟域设计	5
3	模块结构	6
3.1	工程文件	6
3.2	各模块关系图	8
3.3	核心模块详细说明	8
3.3.1	game_top — FPGA 顶层模块	8
3.3.2	vgac — VGA 控制器	9
3.3.3	vga_top — VGA 渲染管线顶层	10
3.3.4	game_logic — 核心游戏逻辑	11
3.3.5	字库实现	15
3.3.6	渲染器模块族	15
4	程序流程	16
4.1	游戏主循环	16
4.2	VGA 渲染流程	19
5	调试过程分析	19
5.1	语法与综合调试	19
5.2	主要调试问题与解决方案	20
5.2.1	问题 1: 玩家飞机位置与移动异常	20
5.2.2	问题 2: 敌机生成和显示不全	20
5.2.3	问题 3: 子弹发射数量少	20
5.2.4	问题 4: 文字显示异常	21
5.2.5	问题 5: 椭球体渲染鬼影点	21
5.3	综合资源使用	21

6	核心模块模拟仿真结果	21
6.1	键盘驱动模块 (Keyboard) 仿真	21
6.2	游戏逻辑模块 (game_logic) 仿真	22
6.3	VGA 渲染管线仿真	23
7	操作说明	23
7.1	游戏启动	23
7.2	菜单操作	24
7.3	游戏操作	24
7.4	游戏机制	24
8	验证过程演示	24
8.1	上板验证流程	24
8.2	上板测试结果	26
9	程序代码 (关键部分)	26
9.1	game_top — 顶层互联	26
9.2	game_logic — 状态机与输入映射	27
9.3	game_logic — FSM 状态转换	27
9.4	LFSR 伪随机数生成	28
9.5	hud_render — 内联字库 (部分)	28
10	组内成员分工说明及贡献比例	29
10.1	成员分工	29
10.2	贡献比例	29
11	总结与展望	30
11.1	项目总结	30
11.2	改进方向	30
12	参考资料	31

1 背景介绍

本课程设计基于 Sword Kintex-7 实验平台（主控芯片 XC7K160TFFG676-1），要求设计一个具有完整功能的时序电路，综合运用有限状态机、寄存器/存储器访问、人机交互 I/O 接口等数字电路设计要素。

本小组选择设计一个基于 FPGA 的弹幕射击游戏。弹幕射击游戏(Danmaku Shooting)是一种经典的视频游戏类型：玩家控制飞行器在屏幕上移动，发射子弹攻击敌方单位，同时躲避敌方子弹。该选题兼具趣味性和技术挑战性，涵盖了 FSM 设计、VGA 显示控制、碰撞检测、伪随机数生成、优先级合成渲染等数字电路设计核心内容。

本报告将详细介绍该游戏的设计思路、系统架构及各模块实现细节。

2 设计说明

2.1 设计开发环境

- 实验平台：Sword Kintex-7开发板（XC7K160TFFG676-1）
- 开发环境：Xilinx Vivado 2025.1
- 硬件描述语言：Verilog HDL
- 系统时钟：100MHz（板载晶振，引脚 AC18）
- 显示输出：VGA 640×480 @ 60Hz，12 位色深（RGB 各 4 位）
- 输入设备：PS/2键盘（通过 Keyboard.v 模块解码）
- 辅助显示：4位 7 段数码管、8 位 LED

2.2 输入输出交互选择

2.2.1 键盘输入

- **PS/2 键盘：**通过 Keyboard.v 模块解码 PS/2 Set 2 协议，输出 8 位按键状态 bitmap 和 4 位难度选择 one-hot 信号。按键映射如下：
 - W/A/S/D：上/左/下/右移动
 - J/Space/Enter：开火/开始游戏
 - K：武器升级
 - P：暂停/恢复
 - 1/2/3/4：选择难度（EASY/NORMAL/HARD/HELL）

- M: 切换模式 (积分/无尽)
- Tab: 切换 7 段数码管显示内容

2.2.2 显示输出

- **VGA 显示器:** 640×480 @ 60Hz, 12 位 BGR 色深, 实时渲染游戏画面
- **4 位 7 段数码管 (直连模式):** 可切换显示击毁敌人数/玩家生命值/当前积分, 通过 DispNum 模块动态扫描驱动
- **4 位 7 段数码管 (串行 P2S 模式):** 通过 Seg7Device + ShiftReg 串行移位输出级联扩展
- **8 位 LED:** 积分高位溢出时用二进制表示高位部分; 串行 P2S 扩展 LED

2.3 系统总体架构

系统自顶向下分为三层, 采用” 逻辑引擎 → 状态数据 → 渲染管线” 的数据流架构:

- (1) **顶层互连层 (game_top):** 负责 32 位自由运行计数器 (100MHz 递增) 的时钟分频、PS/2 键盘解码接入、各子系统例化和 K7 引脚连接
- (2) **核心功能层:**
 - **VGA 渲染管线 (vga_top):** 由 25MHz VGA 时钟驱动 vgac 控制器产生扫描坐标 (row_addr, col_addr); 所有渲染器并行接收坐标, 纯组合逻辑输出该像素的 hit 和颜色; 优先级 MUX 合成最终像素; 同时检测 vs 下降沿生成 60Hz frame_tick 脉冲
 - **游戏逻辑引擎 (game_logic):** 运行于 60Hz frame_tick 域, 管理 5 状态 FSM、实体状态更新 (玩家/敌机/子弹/障碍物)、逐对 AABB 碰撞检测、LFSR 伪随机数生成、武器升级、计分系统
- (3) **外设驱动层:** PS/2 键盘驱动 (Keyboard)、7 段数码管直连显示 (DispNum + SegmentDecoder + Seg7Decode + Seg7Remap)、7 段数码管串行 P2S (Seg7Device + ShiftReg)、LED 串行 P2S (ShiftReg)

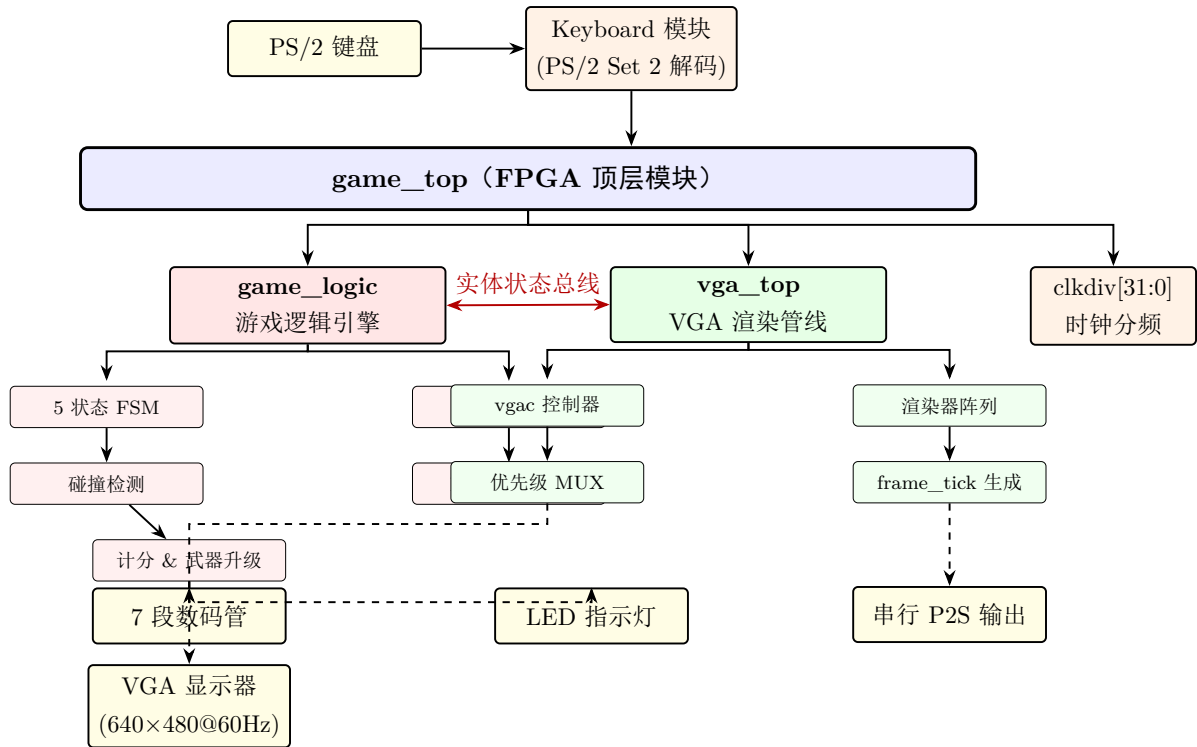


图 1: 系统总体架构框图

2.4 引脚约束

系统通过 K7.xdc 文件定义所有 FPGA 引脚约束，包括：

- **系统时钟**：100MHz 差分时钟，引脚 AC18 (clk)
- **全局复位**：低有效复位，引脚 W13 (rstn)
- **VGA 输出**：R[3:0] (M17/N16/L16/K17)、G[3:0] (J19/J20/K19/K20)、B[3:0] (M20/M19/L17/L18)、HS (M22)、VS (M21)
- **键盘输入**：PS/2 接口 (ps2_clk / ps2_data)
- **7 段数码管**：SEGMENT[7:0]、AN[3:0]
- **串行输出**：SEG_CLK/SEG_DT/SEG_EN/SEG_CLR、LED_CLK/LED_DT/LED_EN/LED_CLR
- **LED**：LED[7:0]

2.5 时钟域设计

系统在 100MHz 主时钟下运行。关键时钟信号：

- **vga_clk = 25MHz**：2 位计数器 cdiv[1:0] 对 100MHz 进行 4 分频产生，驱动 VGA 控制器和所有纯组合渲染器

- **frame_tick (约 60Hz):** 在 100MHz 域中通过三级同步器 (`vs_sync[2:0]`) 检测 vs 信号的下降沿 (`vs_sync[2] && !vs_sync[1]`), 每个 V-blank 期间产生一个周期的脉冲, 驱动游戏逻辑状态更新
- **PS/2 键盘接口:** 标准 PS/2 Set 2 协议解码
- **数码管动态扫描 (约 6.25MHz):** `clkdiv[15:14]` 用于 DispNum 的扫描选择信号
- **串行移位时钟:** `clkdiv[3]` ($100\text{MHz} \div 16 \approx 6.25\text{MHz}$) 用于 ShiftReg 的串行输出

3 模块结构

3.1 工程文件

工程源码共包含 18 个 Verilog 源文件 (.v) 和 1 个头文件 (.vh), 按功能分类如下:

表 1: 工程源文件列表

类别	文件名	功能
头文件	game_defs.vh	全局宏定义（屏幕尺寸、颜色、状态编码、实体数量、游戏参数等）
顶层	game_top.v	FPGA 顶层（时钟分频、PS/2 键盘接入、子系统互联、引脚映射）
	DispNum.v	4 位 7 段数码管动态扫描显示驱动（含 2-4 译码器、多路选择器、MC14495 兼容译码器）
VGA 基础	vgac.v	ZJU VGA 控制器（640×480@60Hz，输出扫描坐标、同步信号、消隐）
	vga_top.v	VGA 渲染管线顶层（25MHz 时钟分频、渲染器例化、优先级 MUX、frame_tick 生成）
渲染器	player_render.v	玩家三角形飞机渲染（纯组合逻辑）
	enemy_render.v	敌机椭圆 UFO 渲染（8 实例，generate 展开）
	bullet_render.v	子弹渲染（16 玩家菱形 + 64 敌弹方块，generate 展开）
	obstacle_render.v	障碍物石头渲染（8 实例）
	menu_render.v	菜单界面渲染（标题/装饰边框/难度/模式/操作提示文字）
游戏逻辑	hud_render.v	HUD 渲染（分数/生命/暂停/GAMEOVER/YOU WIN 文字）
	game_logic.v	核心逻辑（5 状态 FSM、实体管理、碰撞检测、LFSR、计分、武器升级）
	Keyboard.v	PS/2 键盘驱动（Set 2 协议 make/break 解码 + 扩展键码 E0 前缀处理）
外设驱动	SegmentDecoder.v	单 digit 十六进制 →7 段译码（低有效）
	Seg7Decode.v	8 位 7 段译码 + 锁存使能（级联 8 个 SegmentDecoder）

注: Keyboard.v 已实例化到 game_top 顶层中, 所有输入通过 PS/2 键盘完成。

3.2 各模块关系图

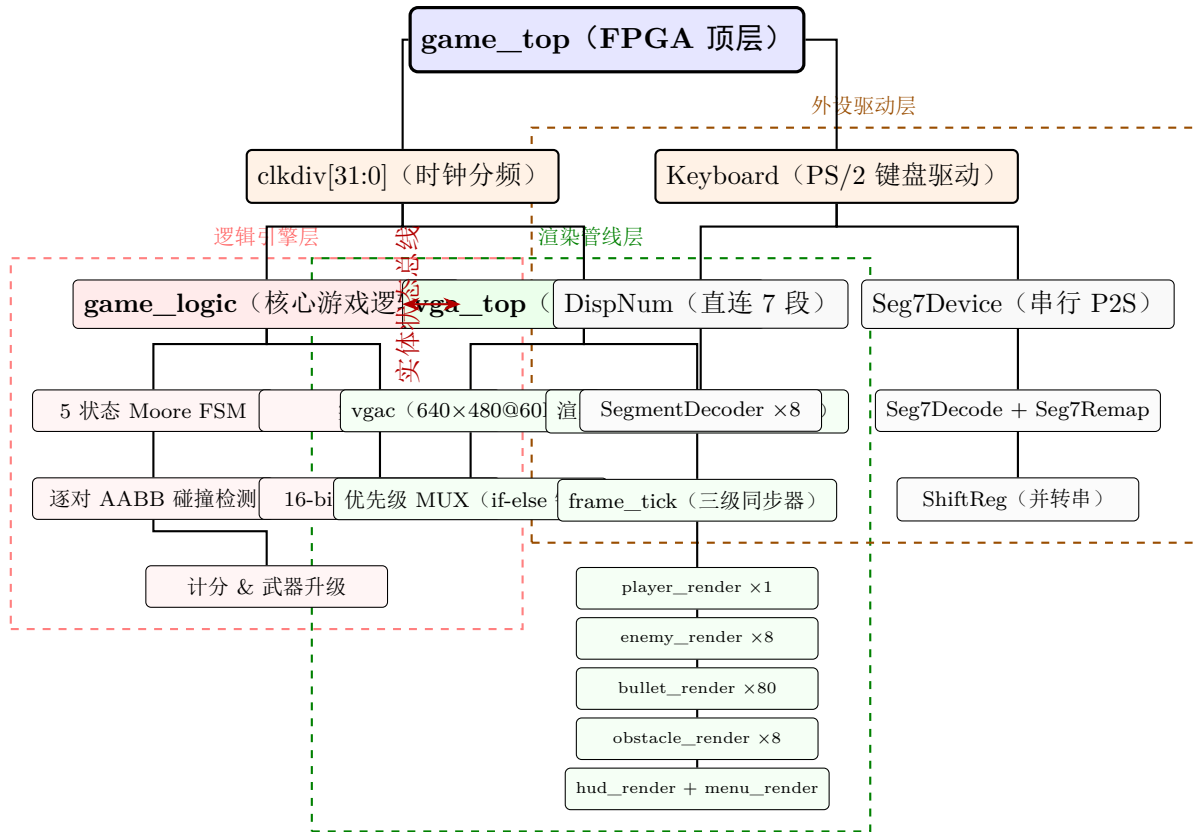


图 2: 模块关系结构图

3.3 核心模块详细说明

3.3.1 game_top — FPGA 顶层模块

game_top 是 FPGA 顶层模块, 端口与 K7.xdc 引脚约束一一对应:

```

1 module game_top (
2     input wire      clk,           // 100MHz 系统时钟
3     input wire      rstn,          // 低有效复位
4     output wire [3:0] r, g, b,     // VGA 色彩通道
5     output wire      hs, vs,       // VGA 行/场同步
6     input wire      ps2_clk,       // PS/2 键盘时钟
7     input wire      ps2_data,      // PS/2 键盘数据
8     output wire [7:0] SEGMENT,     // 7 段数码管段选 (直连)
9     output wire [3:0] AN,          // 7 段数码管位选 (直连)
10    output wire      SEG_CLK, SEG_CLR, SEG_DT, SEG_EN, // 7 段串行 P2S
11    output wire [7:0] LED,          // LED 直连
12    output wire      LED_CLK, LED_CLR, LED_DT, LED_EN // LED 串行 P2S

```

13 |);

顶层内部结构:

- 32 位自由运行计数器 `clkdiv` (100MHz 递增), 用于多级分频
- 例化 `Keyboard` (PS/2 键盘驱动, 输出按键 `bitmap`、难度、模式等状态信号)
- 例化 `vga_top` (VGA 渲染管线, 输出 `r/g/b/hs/vs` 和 `frame_tick`)
- 例化 `game_logic` (核心游戏逻辑, 接收 `frame_tick` 和键盘状态, 输出所有实体状态)
- 例化 `DispNum` (4 位 7 段数码管直连动态扫描显示)
- 例化 `Seg7Device + ShiftReg` (串行 7 段 P2S 和 LED P2S 输出)

【图: `game_top` 模块 RTL Schematic】

图 3: `game_top` 模块 RTL Schematic

3.3.2 `vgac` — VGA 控制器

`vgac` 为标准 $640 \times 480 @ 60\text{Hz}$ VGA 时序控制器, 驱动 ZJU VGA 接口。关键时序参数:

- 水平时序: 一行共计 800 像素时钟周期 (`h_count` 0~799), 有效显示区 144~783 (640 像素), 行同步 `h_count > 95` 时输出高
- 垂直时序: 一帧共计 525 行 (`v_count` 0~524), 有效显示区 35~514 (480 行), 场同步 `v_count > 1` 时输出高
- 地址生成: $\text{row_addr} = \text{v_count} - 35$ (范围 0~479), $\text{col_addr} = \text{h_count} - 143$ (范围 0~639)

- 消隐控制：有效显示区内 $rdn = 0$ （读使能），消隐期间 $rdn = 1$ 强制输出黑色
- 颜色位序： $d_in = \{B[3:0], G[3:0], R[3:0]\}$ （BGR 序，非 RGB 序）

3.3.3 vga_top — VGA 渲染管线顶层

vga_top 负责将游戏数据转换为 VGA 视频信号，是整个渲染系统的顶层：

VGA 时钟：通过 2 位计数器 $cdiv[1:0]$ 对 100MHz 系统时钟 4 分频得到 25MHz 像素时钟。

帧同步脉冲生成：通过三级同步器 $vs_sync[2:0]$ 在 100MHz 域中检测 vs 信号的下降沿（1→0），在 V-blank 期间产生一个周期的 $frame_tick$ 脉冲，频率约 60Hz。这一设计确保了游戏状态更新不会造成画面撕裂。

渲染器阵列：所有渲染器均为纯组合逻辑，在同一 25MHz 像素时钟域中并行工作。每个渲染器接收扫描坐标 (sx, sy) ，输出 hit （当前像素是否属于该实体）和 12 位 BGR 颜色信号。渲染器实例通过 **generate** 块展开：

表 2: 渲染器实例列表

渲染器	实例数	展开方式	说明
player_render	1	单实例	玩家三角形飞机
enemy_render	8	generate for	敌人椭圆 UFO，每个独立坐标和 HP
bullet_render（玩家）	16	generate for	玩 家 菱 形 子 弹， $is_player=1$
bullet_render（敌方）	64	generate for	敌方红色方块子弹， $is_player=0$
obstacle_render	8	generate for	障碍物石头，含大小和形状参数
hud_render	1	单实例	HUD 文字叠加层（分数/HP/暂停/结束）
menu_render	1	单实例	菜 单 界 面（仅在 STATE_MENU 下有效）

优先级多路选择器（Priority MUX）：通过级联的 **if-else** 优先级链实现像素合成。消隐期间（ $rdn=1$ ）强制输出黑色。其余层级按以下顺序从高到低：

Menu（仅 MENU 状态）> HUD > 玩家飞机 > 玩家子弹 > 敌方子弹 > 敌人 > 障碍物 > 黑色背景

优先级 MUX 核心代码：

```

1 always @(*) begin
2     if (rdn)                                vga_data = `COL_BLACK;
3     else if (hit_menu && game_state == `STATE_MENU) vga_data = col_menu;
4     else if (hit_hud)                        vga_data = col_hud;
5     else if (hit_pl)                         vga_data = col_pl;
6     else if (hit_pb)                         vga_data = col_pb;
7     else if (hit_eb)                         vga_data = col_eb;
8     else if (hit_en)                         vga_data = col_en;
9     else if (hit_ob)                         vga_data = col_ob;
10    else                                     vga_data = `COL_BLACK;
11 end

```

【图：vga_top 模块 RTL Schematic】

图 4: vga_top 模块 RTL Schematic

3.3.4 game_logic — 核心游戏逻辑

game_logic 是整个游戏的核心模块，由 frame_tick（约 60Hz）驱动，在 100MHz 时钟域中运行。

游戏状态机 系统采用 5 状态 Moore 型有限状态机，状态编码定义于 game_defs.vh:

- **STATE_MENU** (3'd0): 主菜单界面，允许选择难度和游戏模式，等待开始信号
- **STATE_PLAYING** (3'd1): 游戏进行中，实体状态更新、碰撞检测、计分
- **STATE_PAUSED** (3'd2): 暂停状态，画面冻结，约 4 秒冷却后允许恢复
- **STATE_GAMEOVER** (3'd3): 游戏结束（玩家生命值归零），显示 300 帧（5 秒）后返回 MENU

- **STATE_WIN** (3'd4): 通关 (积分模式达到 500 分), 显示 300 帧 (5 秒) 后返回 MENU

状态转换条件:

- (1) MENU → PLAYING: 在 MENU 状态下检测到 start_key (J/Space/Enter 键) 有效, 记录当前选择的难度和模式, 初始化玩家 HP (默认为 3, HELL 难度为 2), 设置 state_timer = GRACE_PERIOD (300 帧, 约 5 秒) 开局无敌保护期
- (2) PLAYING → PAUSED: 在 PLAYING 状态下, 检测到 pause_key (P 键上升沿) 且 pause_cooldown == 0 时进入暂停。进入后设置 pause_cooldown = 250 (约 4 秒冷却)
- (3) PAUSED → PLAYING: 在 PAUSED 状态下, 检测到 pause_key 或 fire_key 且 pause_cooldown == 0 时恢复游戏, 同时设置冷却防止连按
- (4) PLAYING → GAMEOVER: 在 PLAYING 状态下, pl_hp == 0 (玩家生命归零) 时触发。设置 state_timer = GAMEOVER_DISPLAY (300 帧 = 5 秒)
- (5) PLAYING → WIN: 在 PLAYING 状态下, 积分模式下 total_score >= SCORE_WIN_TARGET (500) 且 pl_alive 为真时触发。设置 state_timer = WIN_DISPLAY (300 帧 = 5 秒)
- (6) GAMEOVER → MENU: state_timer 倒计时至 0 后自动返回, 同时清除所有实体
- (7) WIN → MENU: state_timer 倒计时至 0 后自动返回, 同时清除所有实体

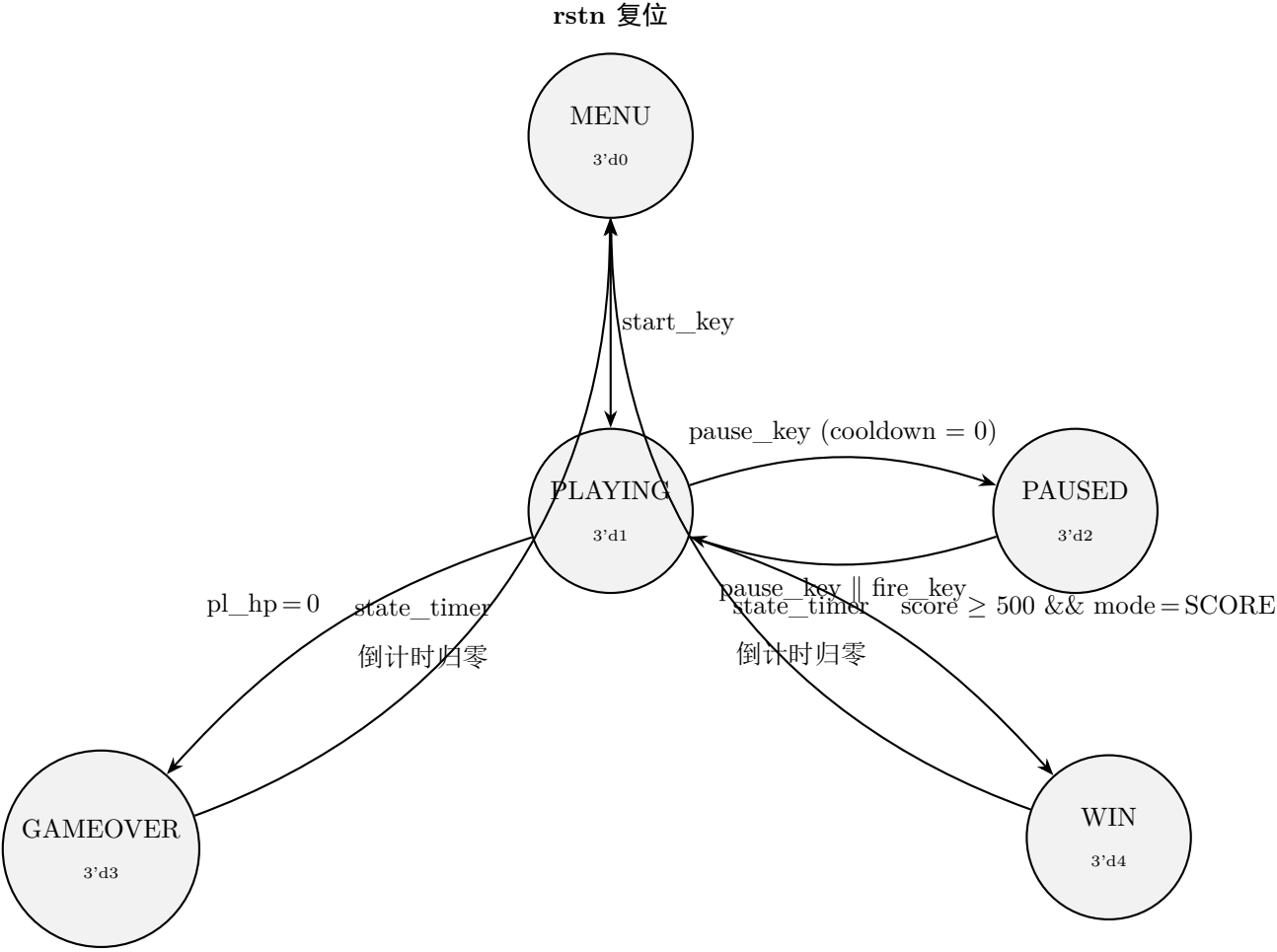


图 5: 游戏状态机状态转换图 (Moore 型 FSM)

输入映射 `game_logic` 通过 `Keyboard.v` 模块获取解码后的键盘输入信号:

表 3: 键盘按键映射表

按键	功能	类型
W/A/S/D	上/左/下/右移动	持续有效
J / Space / Enter	开火 / 开始游戏	持续有效
K	武器升级 (达到击杀阈值时)	持续有效
P	暂停 / 恢复	上升沿触发
1/2/3/4	选择难度 (EASY/NORMAL/HARD/HELL)	即时生效
M	切换模式 (积分/无尽)	上升沿触发
Tab	切换 7 段显示内容	上升沿触发

难度和模式选择在 MENU 状态下通过按键 1-4 和 M 键设定。

实体管理 采用固定槽位池设计，避免动态分配导致的复杂组合逻辑:

- **敌机 (8 槽, en_active[7:0]):** 轮转生成 (en_spawn_idx 0→7→0), LFSR 随机 X 坐标 ($X = \text{lfsr}[5:0] \times 10$), 生成间隔由 ENEMY_SPAWN_BASE (120 帧 = 2 秒) 控制。每架敌机射出 2 发子弹到其专属子槽 (敌机 0→槽 0-1, 敌机 1→槽 8-9, 以此类推, 每敌机独占 8 槽的连续子槽), 仅前 4 架敌机 (索引 0-3) 具有完整的移动和射击逻辑。敌机下移至屏幕底部 ($Y > 470$) 时自动标注 inactive
- **玩家子弹 (16 槽, pb_active[15:0]):** 每次射击优先分配空闲槽。双发模式下分配槽 0 和 1 (左右对称, 偏移 $\pm 5\text{px}$), 武器等级 ≥ 2 时增加槽 2 (中央第三发)。槽 3-15 作为额外通道预留。子弹以 5 像素/帧速度向上移动, 出屏 ($Y \leq 10$) 标记 inactive
- **敌方子弹 (64 槽, eb_active[63:0]):** 每敌机分配专属 8 槽连续空间 (敌机 N→槽 $N*8 \sim N*8+7$), 射击时直接在专属槽写入, 避免搜索空闲槽的深层组合逻辑。子弹以 2 像素/帧速度向下移动, 出屏 ($Y > 470$) 标记 inactive
- **障碍物 (8 槽, ob_active[7:0]):** 轮转生成 (ob_spawn_idx), LFSR 随机参数: 坐标、尺寸 (小/中/大, 由 $\text{lfsr}[15:14]$ 决定)、形状 ($\text{lfsr}[1:0]$ 决定 4 种形状之一)、HP ($\text{lfsr}[13:12] + 1$, 范围 1~4)。生成间隔由 OBSTACLE_SPAWN_BASE (300 帧 = 5 秒) 控制。障碍物以每隔 32 帧 1 像素的速度缓慢下漂, 出屏标注 inactive

碰撞检测 采用逐对 AABB (轴对齐包围盒) 硬编码检测, 无嵌套 for 循环, 确保综合结果可预测:

- 玩家子弹 vs 敌机 (pb0/1/2 vs en0/1/2, 共 9 对): 命中后敌机被击毁 (1 HP 即死), 子弹消失, 总分 +5, 击杀数 +1
- 玩家子弹 vs 障碍物 (pb0/1 vs ob0/1/2/3, 共 4 对): 命中后障碍物消失, 子弹消失, 不计分
- 敌弹 vs 玩家 (eb0/1/8/9, 共 4 对, 需 pl_inv==0 且非作弊模式): 命中后清除附近的敌弹, HP 减 1。若 $HP \leq 1$ 则进入 GAMEOVER; 否则回退至屏幕中央 (320,400), 获得 180 帧无敌保护 (约 3 秒) + 清空所有敌弹
- 玩家 vs 障碍物 (ob0/1): 命中后障碍物消失, 玩家 HP 归零, 进入 GAMEOVER
- 玩家 vs 敌机 (en0/1): 命中后 HP 减 1, 逻辑同敌弹伤害处理

计分与武器升级

- 击毁敌机: +5 分/架 (与敌机 HP 无关)
- 生存奖励: 每 60 帧 (1 秒) surv_cnt 循环 +1 分

- 积分模式目标：500 分通关
- 武器升级：击杀数达到阈值（10/20/40...，每次翻倍）自动升级（最大 3 级），提升子弹数量和缩短射击冷却

玩家属性

- 初始位置：(320, 400)（屏幕中央偏下）
- 移动速度：2 像素/帧，边界限制 $X \in [20, 620]$ 、 $Y \in [20, 455]$
- 无敌时间：受伤后 180 帧（约 3 秒），期间 `pl_flash` 按 `pl_inv[2:1] != 0` 闪烁
- 开火冷却：基础 8 帧，随武器等级缩短为 `FIRE_RATE - weapon`
- 武器升级：击杀 $\geq \text{next_kills_upg}$ （初始 10，每次翻倍）时触发

LFSR 伪随机数生成器 LFSR 实现在 `game_logic.v` 内部（未使用独立模块），16 位 Galois 型，多项式 $x^{16} + x^{14} + x^{13} + x^{11} + 1$ （反馈系数 16'hB400）。每个 game tick 更新一次，种子值 16'hACE1：

```
1 if (tick)
2   lfsr <= {lfsr[14:0], 1'b0} ^ ({16{lfsr[15]}} & 16'hB400);
```

LFSR 输出用于：敌机 X 坐标随机化、敌机射击间隔随机扰动、敌机移动方向选择、障碍物大小/形状/HP 随机化。

3.3.5 字库实现

本工程在 `menu_render.v` 和 `hud_render.v` 中使用 Xilinx Distributed Memory Generator 生成的 ROM IP (`ROM_f`) 实现 8×8 字库，通过 `i.coe` 初始化文件（同目录下）提供位图数据。ROM 地址格式为 `{char_code[5:0], row[2:0]}`（9 位，512 深度），8 位数据输出对应一行像素。

该方案替代了旧版的 `case` 组合逻辑查找表方案，在面积和时序上均有优势。但 `hud_render.v` 中仍保留了一组 `case` 查找表用于特定字符的 inline 字库。

支持的字符包括 A~Z（编码 0~25）、0~9（编码 26~35）、空格（36）、冒号（37）、竖线（39）、短横（40）、大于号（41）、句点（42）。

3.3.6 渲染器模块族

所有渲染器 (`player_render`, `enemy_render`, `bullet_render`, `obstacle_render`, `hud_render`, `menu_render`) 均为纯组合逻辑（无 `posedge`），基于扫描坐标 (`sx`, `sy`) 实时判断每个像素是否属于对应实体。渲染器无时钟输入端，输出 `hit`（1 位）和 12 位 BGR 颜色信号（12 位）。

各渲染器的主要几何特征：

- **玩家 (player_render)**: 向上等腰三角形飞机, 顶点 (px, py-12)、底角 (px±16, py+12)。细节分层: 引擎发光 (橙色)、驾驶舱 (深蓝)、机翼 (白色)、主体 (青色)。支持无敌闪烁 (flash=0 时不可见)
- **敌机 (enemy_render)**: 椭圆 UFO, 简化近似公式 $dx^2 + 2 \cdot dy^2 \leq 200$ (a 14, b 10)。细节: 舷窗 (小方形窗口)、穹顶 (椭圆弧线)、边缘轮廓 (2 像素宽环)。HP 分级着色 (3→ 品红、2→ 紫色、1→ 暗红), 击中闪烁为白色
- **子弹 (bullet_render)**: 通过 is_player 参数区分。玩家弹: 曼哈顿距离 ≤ 2 的菱形 (青色), 敌弹: 3×3 方块 (红色)
- **障碍物 (obstacle_render)**: 根据 size 和 shape 渲染不同大小的石头形状, 4 种几何造型 + 3 种尺寸组合
- **菜单 (menu_render)**: 双层装饰边框 (彩虹色交替 + 浅灰细线)、标题文字 (AEROPLANE / DANMAKU SHOOTING)、START 提示框、难度和模式显示、操作说明
- **HUD (hud_render)**: 右下角 SCORE 显示 (14 字符)、左上角 HP 显示 (4 字符)、暂停 |||| 符号 (黄色)、居中大号 GAME OVER (红色) / YOU WIN (绿色)

4 程序流程

4.1 游戏主循环

系统在 100MHz 时钟域运行。vga_top 通过三级同步器检测 vs 场同步的下降沿, 在 V-blank 期间产生一个 100MHz 时钟周期的 frame_tick 脉冲 (约 60Hz)。game_logic 在每个 frame_tick 上升沿执行一帧游戏逻辑更新。这一机制确保了 VGA 画面渲染与游戏逻辑的严格同步, 避免画面撕裂。

每帧 (frame_tick 脉冲) 执行以下步骤:

- (1) 更新 LFSR 伪随机数寄存器
- (2) 检测 P/Tab 键边沿触发, 更新 pause_key / btn_cyc 锁存器
- (3) 处理 pause_cooldown 递减
- (4) 根据当前 state 执行对应逻辑:

MENU 状态 初始化所有实体寄存器 → 检测按键 1-4 设定难度 → 检测 M 键设定模式 → 等待 start_key 进入 PLAYING (写入难度、模式、初始 HP, 设置 GRACE_PERIOD 开局保护)

PLAYING 状态

- game_time 递增, state_timer 递减
- surv_cnt 循环累计 (60 帧 +1 生存分)
- 玩家移动 (受边界 20~620 / 20~455 限制, 速度 2px/帧)
- 无敌计时 (pl_inv 递减、pl_flash 闪烁控制)
- 玩家射击 (空闲槽分配、冷却检查、武器等级决定子弹数)
- 移动玩家子弹 (向上 5px/帧, 出屏标记 inactive)
- 敌机生成 (轮转空闲槽, LFSR 随机 X, 生成间隔递减)
- 敌机下漂 + 左右微移 + 敌机专属槽射击
- 移动敌方子弹 (向下 2px/帧, 出屏标记 inactive)
- 障碍物生成 (轮转空闲槽, LFSR 随机参数)
- 障碍物缓慢下漂 (每 32 帧 1px)
- 碰撞检测 (逐对 AABB) → 扣血/击毁/得分/无敌保护
- 武器升级检查 ($\text{total_kills} \geq \text{next_kills_upg}$)
- 检查 GAMEOVER (HP=0) 或 WIN ($\text{score} \geq 500$ 且积分模式)

PAUSED 状态 不更新时间, 等待 pause_key 或 fire_key 恢复 (带冷却)

GAMEOVER/WIN 状态 state_timer 倒计时 (300 帧 → 5 秒) → 返回 MENU, 清除所有实体

(5) 更新 7 段数码管显示数据 (根据 seg_cycle 选择击杀/生命/积分)

(6) 更新 LED 溢出显示 (积分 ≥ 10000 或击杀 ≥ 100 时点亮高位 LED)

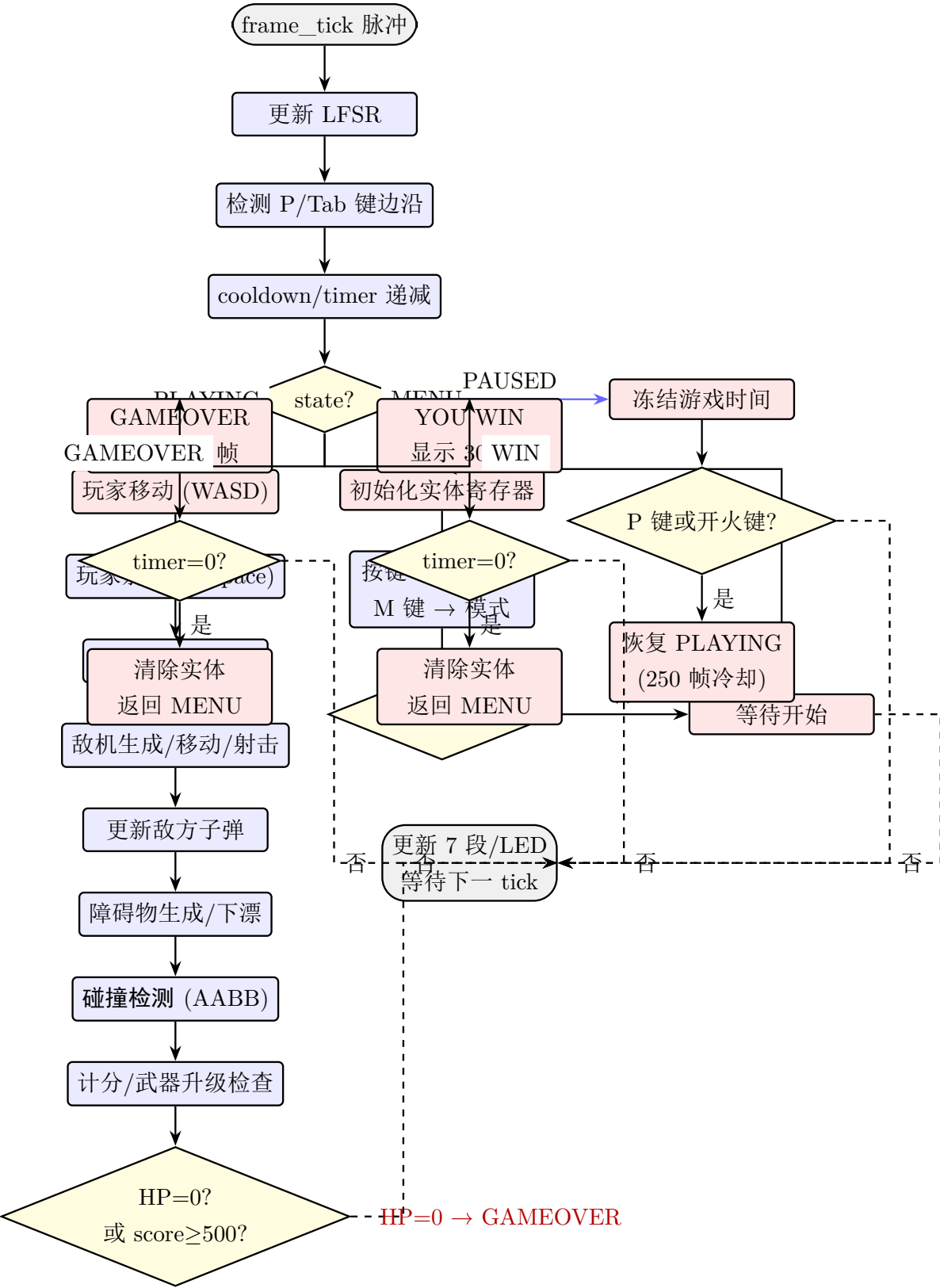


图 6: 游戏主循环整体流程图 (60Hz frame_tick 驱动)

【图：Vivado Synthesis 完成截图 - 资源使用概览】

图 8: Vivado Synthesis 综合完成截图

5.2 主要调试问题与解决方案

5.2.1 问题 1: 玩家飞机位置与移动异常

现象: 玩家飞机初始位置不在屏幕预期位置（中央偏下），移动范围异常。

原因: 玩家初始坐标设置错误，边界检查条件不完整。

解决方案: 将玩家初始位置设为 (320, 400)（对应 640×480 屏幕中央偏下），移动边界限制为 $X \in [20, 620]$ 、 $Y \in [20, 455]$ ，考虑飞机半宽（16px）和半高（12px）。

5.2.2 问题 2: 敌机生成和显示不全

现象: 仅有少数敌机槽（索引 0-3）被更新，其余槽位无响应。

原因: 敌机生成、移动和射击逻辑只针对前 4 个槽位硬编码，未覆盖全部 8 个槽位。

解决方案: 将敌机的生成（en_spawn_idx 轮转覆盖 0~7）、移动和射击逻辑扩展到全部 8 个敌机槽。前 4 槽保持完整 AI 逻辑，后 4 槽提供基础的生成和下移功能。

5.2.3 问题 3: 子弹发射数量少

现象: 玩家每次只能发射 1~2 颗子弹。

原因: 子弹分配仅检查固定槽位（0 和 1），未充分利用 16 个槽位，且未实现武器升级的第三发。

解决方案: 改为优先轮询空闲槽位分配。基础双发分配到槽 0 和 1（左右对称），武器等级 ≥ 2 时增加第三发到槽 2（中央发射）。槽 3-15 作为预留扩展通道。

5.2.4 问题 4：文字显示异常

现象：HUD 和菜单中部分字符（冒号、竖线等）显示为空白或乱码。

原因：字符编码寄存器位宽为 [4:0]（5 位），数值 ≥ 32 的编码（如空格 36、冒号 37、竖线 39）被截断。

解决方案：将字符编码变量位宽从 [4:0] 扩展为 [5:0]，相关常量从 5'dN 改为 6'dN。同时将字库从 inline case 查找表升级为 Xilinx Distributed Memory ROM IP，以 i.coe 初始化文件提供位图数据，提升综合效率。

5.2.5 问题 5：椭圆渲染鬼影点

现象：敌机 UFO 的椭圆渲染在距敌机约 256 像素处产生周期性鬼影点（伪像）。

原因：椭圆距离平方计算 ($dx \cdot dx + 2 \cdot dy \cdot dy$) 的值在远离实体时超过位宽上限导致乘法溢出。具体为 adx2 和 ady2 使用 16 位宽，而 adx^2 理论最大值为 $640^2 = 409600$ （需 19 位）。

解决方案：将 adx2 扩展为 22 位宽、ady2 为 22 位、dist2 为 23 位，阈值比较字面量统一为 23'd 宽度。

5.3 综合资源使用

表 4: 资源使用情况（以 Vivado 综合报告为准）

资源	已使用	总量	利用率
Slice LUTs	[待填入]	101,400	[%]
Slice Registers	[待填入]	202,800	[%]
Block RAM	[待填入]	325	[%]
IO	[待填入]	400	[%]

6 核心模块模拟仿真结果

6.1 键盘驱动模块（Keyboard）仿真

对 Keyboard 模块进行仿真，验证 PS/2 Set 2 协议的 make/break 解码逻辑：模块接收 PS/2 时钟和数据信号，解码输出 8 位按键状态 bitmap 和 4 位难度 one-hot 信号。键盘按下时产生 make 码（含 E0 扩展前缀处理），松开时产生 break 码（F0 前缀 + make 码）。

【图：Keyboard 仿真波形 - 展示 PS/2 解码效果】

图 9: Keyboard 模块仿真波形

6.2 游戏逻辑模块 (game_logic) 仿真

对 game_logic 模块进行仿真，验证 FSM 状态转换、实体更新、碰撞检测、计分系统等。

主要验证项：

- MENU → PLAYING 状态转换 (start_key 触发)
- PLAYING → PAUSED → PLAYING 暂停恢复循环
- 玩家移动边界限制 (X 20~620, Y 20~455)
- 玩家子弹发射与槽位分配逻辑
- 敌机轮转生成、移动与射击
- AABB 碰撞检测判定
- HP 扣减与无敌保护 (pl_inv 倒计时 + pl_flash 闪烁)
- GAMEOVER 触发条件 (HP = 0)
- WIN 触发条件 (积分模式 score ≥ 500)
- 武器升级检查 (total_kills ≥ next_kills_upg)

【图：game_logic 仿真波形 - 展示状态转换和关键信号】

图 10: game_logic 模块仿真波形

6.3 VGA 渲染管线仿真

上板实测为主，仿真时重点检查 vga_top 的 RGB 数据时序和同步信号。

【图：vga_top 仿真波形 - 展示 VGA 时序和像素输出】

图 11: vga_top VGA 渲染管线仿真波形

7 操作说明

7.1 游戏启动

- (1) 将.bit文件通过 Vivado Hardware Manager 下载到 K7 FPGA 开发板
- (2) 连接 VGA 显示器到开发板 VGA 接口
- (3) 按下全局复位键（rstn），系统进入主菜单界面

7.2 菜单操作

- 按键 1~4: 选择难度 (EASY / NORMAL / HARD / HELL)
- 按键 M: 切换模式 (积分 / 无尽)
- 按下 J / Space / Enter: 开始游戏 (前 5 秒为开局保护期, 不刷新敌机和障碍物)

7.3 游戏操作

- W/A/S/D: 上/左/下/右移动玩家飞机
- J/Space/Enter: 发射子弹 (按住连发, 受冷却限制)
- P: 暂停/继续 (单次触发, 约 4 秒冷却)
- Tab: 切换 7 段数码管显示内容 (击毁数 → 生命值 → 积分 → 循环)
- K: 武器升级 (达到击杀阈值时按下)

7.4 游戏机制

- 默认 3 条生命 (HELL 难度为 2)
- 击毁敌机 +5 分, 每秒生存奖励 +1 分
- 积分模式达到 500 分显示 YOU WIN, 5 秒后返回菜单
- 无尽模式无通关条件, 尽量延长存活时间
- 生命归零显示 GAME OVER, 5 秒后返回菜单
- 受伤后 180 帧 (约 3 秒) 无敌保护, 飞机闪烁提示
- 击杀数达 10/20/40 依次升级武器 (单发 → 双发 → 三发)

8 验证过程演示

8.1 上板验证流程

- (1) Vivado 完成综合 (Synthesis) → 实现 (Implementation) → 生成 .bit 文件 (Generate Bitstream)
- (2) 通过 Hardware Manager → Program Device 下载 .bit 到 K7 开发板

- (3) 连接 VGA 显示器，确认显示正常
- (4) 按下全局复位键 (rstn)，检查主菜单画面
- (5) 切换难度 (按键 1-4) 和模式 (M 键)，验证菜单文字实时变化
- (6) 按下开始键 (J/Space/Enter) 进入游戏，验证：
 - 前 5 秒不刷新敌机和障碍物
 - 玩家飞机移动范围和速度 (WASD)
 - 子弹发射 (双发 + 武器升级后的三发)
 - 敌机生成、移动和射击
 - 障碍物生成和漂移
 - 碰撞检测：玩家子弹击中敌机 (得分 +5)、击中障碍物 (消失)
 - 玩家受伤：HP-1、无敌闪烁、弹幕清空
 - HP 归零 → GAMEOVER 画面
 - 积分 ≥ 500 (积分模式) → YOU WIN 画面
- (7) 验证暂停/恢复 (P 键)
- (8) 验证 7 段数码管显示切换 (Tab 键)
- (9) 验证武器升级 (K 键，达到击杀阈值后)

8.2 上板测试结果

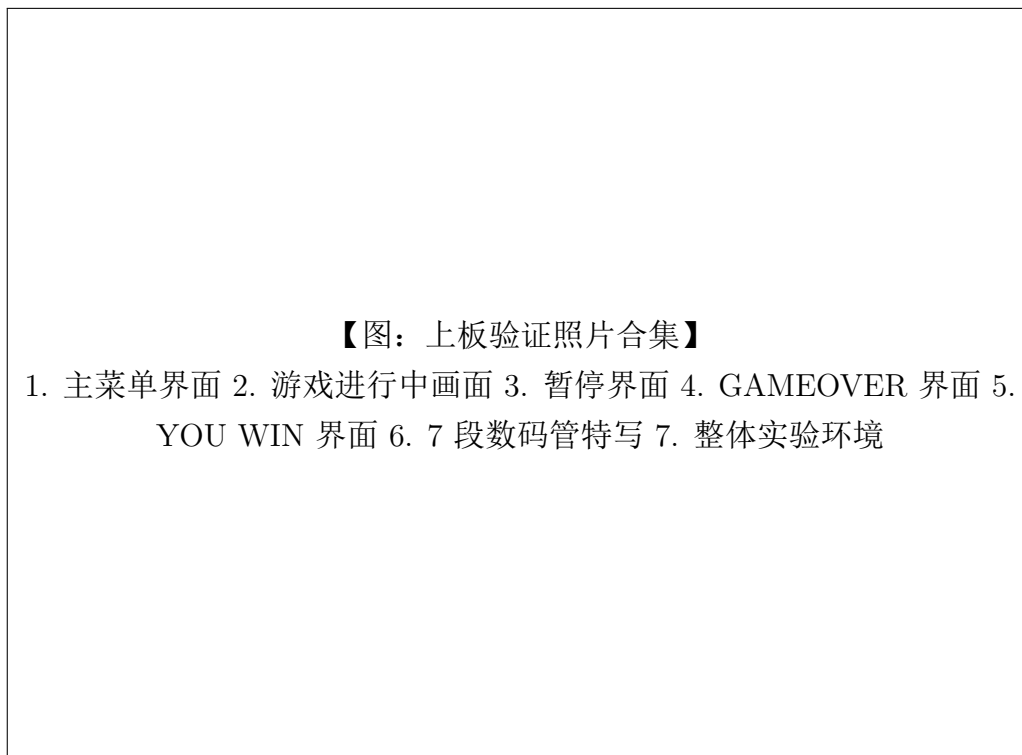


图 12: 上板验证结果

9 程序代码（关键部分）

9.1 game_top — 顶层互联

```
1 // 32 位自由运行计数器（100MHz 时钟分频源）
2 reg [31:0] clkdiv;
3 always @(posedge clk) clkdiv <= clkdiv + 1'b1;
4
5 // PS/2 键盘驱动例化
6 wire [7:0] key_data;           // 按键状态 bitmap
7 wire [3:0] key_diff;          // 难度选择 one-hot
8 wire      key_mode;           // 模式切换
9 wire      key_cycle;          // 显示切换
10
11 Keyboard u_keyboard (
12     .clk(clk), .rstn(rstn),
13     .ps2_clk(ps2_clk), .ps2_data(ps2_data),
14     .key_data(key_data), .key_diff(key_diff),
15     .key_mode(key_mode), .key_cycle(key_cycle)
16 );
```

9.2 game_logic — 状态机与输入映射

```

1 // Keyboard 模块输出:
2 // key_data[0]=W, key_data[1]=A, key_data[2]=S, key_data[3]=D
3 // key_data[4]=J, key_data[5]=Space, key_data[6]=Enter, key_data[7]=K
4 wire move_up    = key_data[0];    // W
5 wire move_left  = key_data[1];    // A
6 wire move_down  = key_data[2];    // S
7 wire move_right = key_data[3];    // D
8 wire fire_key   = key_data[4] || key_data[5] || key_data[6]; // J/Space/
   Enter
9 wire start_key  = fire_key;
10 wire upgrade   = key_data[7];    // K

```

9.3 game_logic — FSM 状态转换

```

1 case (state)
2 `STATE_MENU: begin
3     // 初始化所有实体寄存器
4     pl_alive <= 1'b1; pl_x <= 10'd320; pl_y <= 9'd400;
5     total_score <= 20'd0; total_kills <= 8'd0;
6     en_act <= 8'd0; pb_act <= 16'd0; eb_act <= 64'd0; ob_act <= 8'd0;
7
8     // 难度与模式选择
9     if (key_diff[0]) sel_diff <= `DIFF_EASY;    // 按键 1
10    if (key_diff[1]) sel_diff <= `DIFF_NORMAL;  // 按键 2
11    if (key_diff[2]) sel_diff <= `DIFF_HARD;    // 按键 3
12    if (key_diff[3]) sel_diff <= `DIFF_HELL;    // 按键 4
13    sel_mode <= key_mode ^ sel_mode;            // M 键切换
14
15    if (start_key) begin
16        state <= `STATE_PLAYING;
17        state_timer <= `GRACE_PERIOD; // 300 帧 = 5 秒开局保护
18        difficulty <= sel_diff;
19        mode <= sel_mode;
20        pl_hp <= (sel_diff == `DIFF_EASY) ? 3'd3 :
21                (sel_diff == `DIFF_HELL) ? 3'd2 : 3'd3;
22    end
23 end
24
25 `STATE_PLAYING: begin
26     // 暂停检测 (带冷却)
27     if (pause_key && pause_cooldown == 8'd0) begin
28         state <= `STATE_PAUSED;

```

```

29     pause_cooldown <= 8'd250; // 约 4 秒冷却
30 end
31
32 // ..... 实体更新、碰撞检测、计分 .....
33
34 // 状态转移检测
35 if (pl_hp == 0) begin
36     state <= `STATE_GAMEOVER;
37     state_timer <= `GAMEOVER_DISPLAY;
38 end else if (!mode && total_score >= `SCORE_WIN_TARGET) begin
39     state <= `STATE_WIN;
40     state_timer <= `WIN_DISPLAY;
41 end
42 end
43
44 `STATE_PAUSED: begin
45     if ((pause_key || fire_key) && pause_cooldown == 8'd0) begin
46         state <= `STATE_PLAYING;
47         pause_cooldown <= 8'd250;
48     end
49 end
50
51 `STATE_GAMEOVER: begin
52     if (state_timer > 0) state_timer <= state_timer - 11'd1;
53     else state <= `STATE_MENU; // 5 秒后返回菜单
54 end
55
56 `STATE_WIN: begin
57     if (state_timer > 0) state_timer <= state_timer - 11'd1;
58     else state <= `STATE_MENU; // 5 秒后返回菜单
59 end
60 endcase

```

9.4 LFSR 伪随机数生成

```

1 // 16 位 Galois LFSR, 多项式  $x^{16} + x^{14} + x^{13} + x^{11} + 1$ 
2 // 反馈系数: 16'hB400 = 16'b1011_0100_0000_0000
3 // 种子值: 16'hACE1
4 if (tick)
5     lfsr <= {lfsr[14:0], 1'b0} ^ ({16{lfsr[15]}} & 16'hB400);

```

9.5 hud_render — 内联字库（部分）

```
1 // 字符 ROM 综合为 Distributed Memory ROM IP (ROM_f)
2 // 地址格式: {char_code[5:0], sy[2:0]} = 9 位, 512 深度
3 // 数据位宽: 8 位 (一行像素的位图)
4 ROM_f u_font_rom (
5     .a    ({char_code[5:0], sy[2:0]}),
6     .spo(char_bitmap)
7 );
8 // 字库初始化文件: i.coe (项目根目录)
```

10 组内成员分工说明及贡献比例

10.1 成员分工

表 5: 成员分工表

姓名	负责内容	具体工作
[姓名 1] (组长)	系统架构、VGA 渲染管线	整体架构设计、game_top 顶层互联、VGA 渲染管线 (vga_top + vgac + 优先级 MUX)、渲染器族 (player/enemy/bullet/menu/hud_render)、字库 ROM、报告撰写
[姓名 2]	游戏逻辑、外设驱动	game_logic 核心 FSM、实体管理与碰撞检测、LFSR 随机数、PS/2 键盘驱动 (Keyboard)、7 段数码管驱动、上板调试
[姓名 3]	仿真验证、操作说明	各模块单元仿真、上板验证测试、操作说明书编写、实验报告排版

10.2 贡献比例

- [姓名 1] (组长): 34%
- [姓名 2]: 33%
- [姓名 3]: 33%

11 总结与展望

11.1 项目总结

本工程在 Sword Kintex-7 FPGA 平台上实现了一个完整的弹幕射击游戏，涵盖以下数字逻辑设计要点：

- (1) **有限状态机(FSM)设计**: 5状态 Moore 型状态机(MENU/PLAYING/PAUSED/GAMEOVER/VICTORY)，完整的状态转换条件判断、边沿触发锁存消费机制和冷却保护
- (2) **VGA 显示控制**: 640×480 @ 60Hz, 12 位 BGR 色深, 纯组合逻辑渲染器族 + 优先级合成管线, 多实体并行渲染无帧缓冲设计
- (3) **人机交互接口**: PS/2键盘输入 (Keyboard 模块解码), VGA 视频 + 7 段数码管 (直连 + 串行 P2S) + LED 多种输出
- (4) **实体系统管理**: 8敌机 + 16 玩家子弹 + 64 敌方子弹 + 8 障碍物, 固定槽位池管理 + 专属子槽分配, 通过扁平化高位宽总线与 generate 展开在综合层面优化
- (5) **碰撞检测**: 逐对 AABB 轴对齐包围盒硬编码检测, 无嵌套 for 循环, 多类型交互 (玩家子弹 × 敌机、玩家子弹 × 障碍物、敌弹 × 玩家、玩家 × 敌机、玩家 × 障碍物)
- (6) **伪随机数生成**: 16位 Galois LFSR, 多项式 $x^{16} + x^{14} + x^{13} + x^{11} + 1$, 驱动敌机坐标、射击间隔、移动方向、障碍物参数的随机化
- (7) **PS/2 键盘解码**: Keyboard 模块实现 PS/2 Set 2 协议 make/break 解码、E0 扩展前缀处理、8 位按键 bitmap + 4 位难度 one-hot 输出
- (8) **时钟域设计**: 100MHz系统时钟 → 25MHz VGA 时钟 + 60Hz frame_tick 通过三级同步器实现跨时钟域 vs 下降沿检测
- (9) **字库与 ROM**: Xilinx Distributed Memory ROM IP 实现 8×8 字库, i.coe 初始化文件提供位图数据, 支持 A-Z、0-9 和标点共 40+ 字符

11.2 改进方向

- (1) **音效系统**: 开发板蜂鸣器 (Buzzer) 可接入简单 PWM 驱动, 实现射击音效、爆炸音效、背景音乐 (BGM) 等音频反馈
- (2) **曲线弹道**: 当前敌方子弹均为直线弹道 (BTTYPE_STRAIGHT), game_defs.vh 已预留抛物线 (BTTYPE_PARABOLA)、顺时针圆弧 (BTTYPE_CIRCLE_CW)、

逆时针圆弧 (BTTYPE_CIRCLE_CCW) 三种曲线类型编码, 可在 `game_logic` 中实现对应的运动方程更新

- (3) **敌机 AI 多样化:** 当前仅前 4 个敌机槽有差异化行为 (下漂 + 左右微移), 可在全部 8 个槽位实现不同 AI 策略 (悬停射击、S 形移动、追踪玩家、弹幕散布等)
- (4) **图形质量提升:** 当前玩家飞机和敌机 UFO 为几何图形 (三角形/椭圆), 可通过 Block RAM 存储精灵位图 (sprite) 实现更精细的飞机和 UFO 造型
- (5) **难度动态调整:** `game_defs.vh` 已定义难度系数 ($DMUL_EASY = 1.0$ 、 $DMUL_NORMAL = 1.5$ 、 $DMUL_HARD = 2.0$ 、 $DMUL_HELL = 3.0$) 和时间增长函数 $T(t)$ 的框架, 可在 `game_logic` 中实现随时间动态调整敌机刷新闻隔、HP 和子弹密度

12 参考资料

- [1] NJU 数字逻辑设计实验讲义, PS/2 键盘输入实验, <https://nju-projectn.github.io/dlco-lecture-note/exp/07.html>
- [2] YooLc/FPGA-STG: 基于 FPGA 的 STG 射击游戏, <https://github.com/YooLc/FPGA-STG>
- [3] Xilinx, “Vivado Design Suite User Guide: Synthesis,” UG901
- [4] Xilinx, “7 Series FPGAs Data Sheet: Overview,” DS180
- [5] Xilinx, “Vivado Design Suite User Guide: I/O and Clock Planning,” UG899
- [6] J. Bhasker 著, 徐振林等译, “Verilog HDL 硬件描述语言”, 机械工业出版社, 2000
- [7] 夏宇闻, “Verilog 数字系统设计教程”, 北京航空航天大学出版社